```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay

# 1. Load and Prepare Data
iris = load_iris()
X = StandardScaler().fit_transform(iris.data)
Y = OneHotEncoder(sparse_output=False).fit_transform(iris.target.reshape(-1, 1))

X_train, X_test, Y_train, Y_test, y_train, y_test = train_test_split(
    X, Y, iris.target, test_size=0.25, random_state=42, stratify=iris.target
)

# 2. Initialize Network Parameters
np.random.seed(42)
W1, b1 = np.random.randn(X_train.shape[1], 8) * 0.1, np.zeros((1, 8))
W2, b2 = np.random.randn(8, Y_train.shape[1]) * 0.1, np.zeros((1, Y_train.shape[1]))
lr, epochs = 0.05, 1000

def sigmoid(x):
    return 1 / (1 + np.exp(-np.clip(x, -500, 500)))

def softmax(x):
    exp_x = np.exp(x - np.max(x, axis=1, keepdims=True))
    return exp_x / np.sum(exp_x, axis=1, keepdims=True)

# 3. Train via Backpropagation
loss_history = []
for epoch in range(epochs):
    z1 = X_train @ W1 + b1
    a1 = sigmoid(z1)
    z2 = a1 @ W2 + b2
    a2 = softmax(z2)

    loss = -np.mean(np.sum(Y_train * np.log(a2 + 1e-9), axis=1))
    loss_history.append(loss)

    m = len(X_train)
    dz2 = (a2 - Y_train) / m
    dW2, db2 = a1.T @ dz2, np.sum(dz2, axis=0, keepdims=True)

    dz1 = (dz2 @ W2.T) * a1 * (1 - a1)
    dW1, db1 = X_train.T @ dz1, np.sum(dz1, axis=0, keepdims=True)
```

```
    W2 -= lr * dW2
    b2 -= lr * db2
    W1 -= lr * dW1
    b1 -= lr * db1

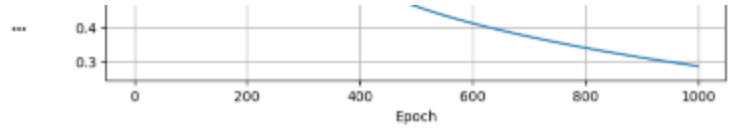
    if (epoch + 1) % 200 == 0:
        print(f"Epoch {epoch+1}, Loss: {loss:.4f}")

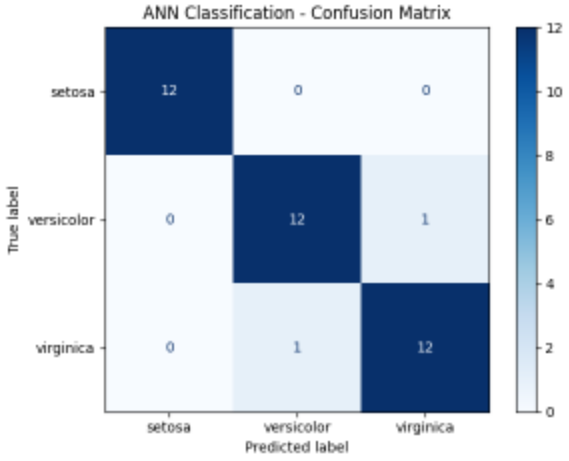
# 4. Test and Evaluate
hidden = sigmoid(X_test @ W1 + b1)
probabilities = softmax(hidden @ W2 + b2)
predictions = np.argmax(probabilities, axis=1)

print(f"\nTest Accuracy: {accuracy_score(y_test, predictions) * 100:.2f}%\n\nSample Predictions:")
for i in range(5):
    print(f"Actual: {iris.target_names[y_test[i]]:10s} | Predicted: {iris.target_names[predictions[i]]}")

# 5. Plots
plt.figure(figsize=(8, 4))
plt.plot(loss_history)
plt.title("ANN Training Using Backpropagation")
plt.xlabel("Epoch")
plt.ylabel("Cross-Entropy Loss")
plt.grid(True)
plt.show()

ConfusionMatrixDisplay.from_predictions(y_test, predictions, display_labels=iris.target_names, cmap="Blues")
plt.title("ANN Classification - Confusion Matrix")
plt.tight_layout()
plt.show()
```





```
Epoch 200, Loss: 0.8975
Epoch 400, Loss: 0.5348
Epoch 600, Loss: 0.4126
Epoch 800, Loss: 0.3407
Epoch 1000, Loss: 0.2872

Test Accuracy: 94.74%

Sample Predictions:
Actual: setosa     | Predicted: setosa
Actual: versicolor | Predicted: versicolor
Actual: versicolor | Predicted: versicolor
Actual: versicolor | Predicted: versicolor
Actual: setosa     | Predicted: setosa
```

